Assignment-6.3

Name: D. Naveen
2403A53L01
Batch:46

Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals

Lab Objectives

- To explore AI-powered auto-completion features for core Python constructs such as classes, loops, and conditional statements.
- To analyze how AI tools suggest logic for object-oriented programming and control structures.
- To evaluate the correctness, readability, and completeness of AI-generated Python code.

Lab Outcomes (LOs)

After completing this lab, students will be able to:

- Use AI tools to generate and complete Python class definitions and methods.
- Understand and assess AI-suggested loop constructs for iterative tasks.
- Generate and evaluate conditional statements using AI-driven prompts.
- Critically analyze AI-assisted code for correctness, clarity, and efficiency.

Task Description #1: Classes (Student Class) Scenario

You are developing a simple student information management module.

Task

- Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
- The class should include attributes such as name, roll number, and branch.
- Add a method display_details() to print student information.
- Execute the code and verify the output.
- Analyze the code generated by the AI tool for correctness and clarity.

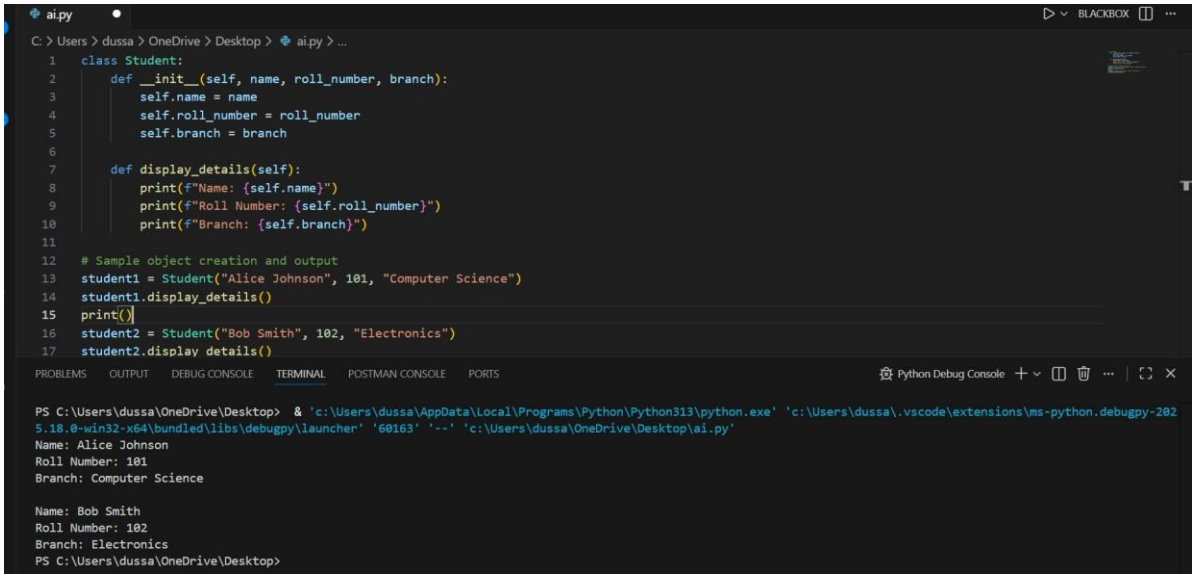
Expected Output #1

- A Python class with a constructor (__init__) and a display_details() method.
- Sample object creation and output displayed on the console.
- Brief analysis of AI-generated code

Prompt:

Generate a Python Student class with attributes name, roll number, and branch. Include a method to display student details. Also show sample object creation and output**.**

Code:

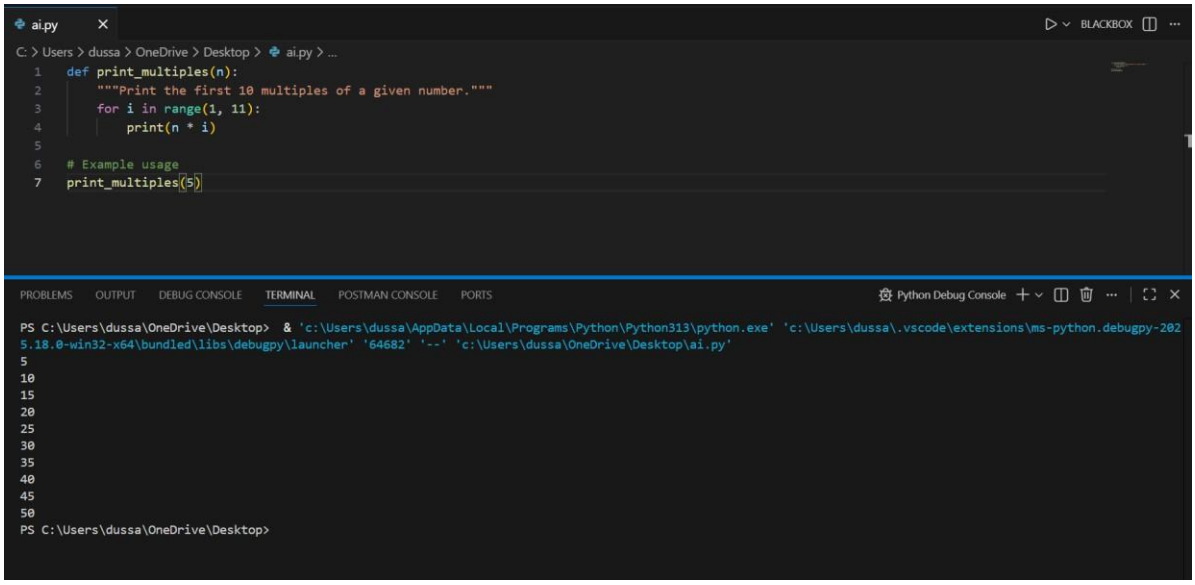


Analysis:
The AI-generated class is correct and readable. It properly uses a constructor (__init__) to initialize attributes and a method to display details. Variable names are clear and follow Python conventions.

Task Description #2: Loops (Multiples of a Number) Scenario
You are writing a utility function to display multiples of a given number.
Task
• Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a
loop.
• Analyze the generated loop logic.
• Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of
for).
Expected Output #2
• Correct loop-based Python implementation.
• Output showing the first 10 multiples of a number.
• Comparison and analysis of different looping approaches

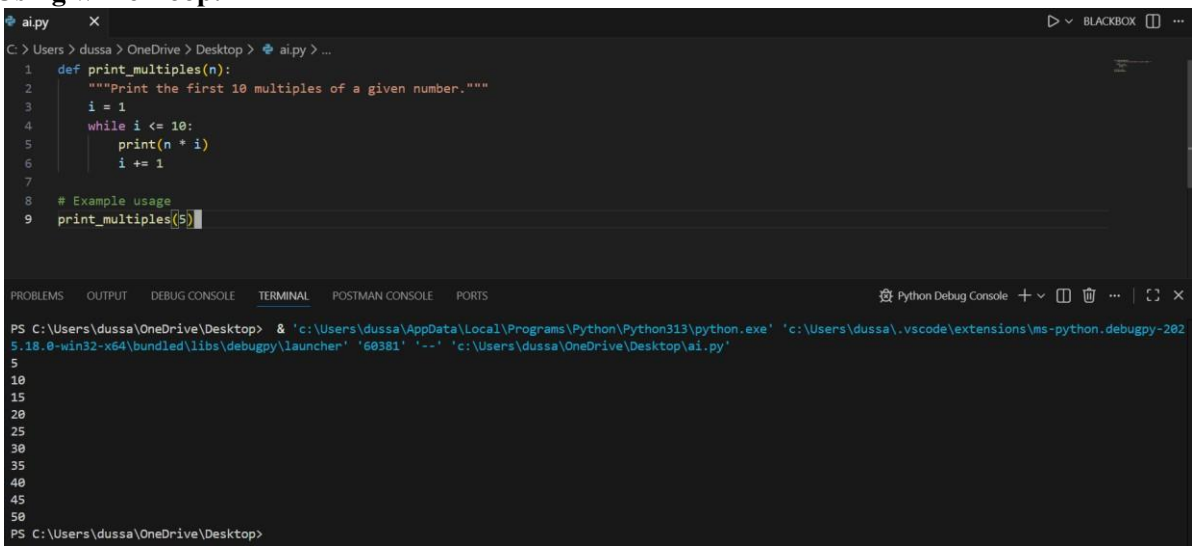
Prompt:
Write a Python function to print the first 10 multiples of a given number using a for loop.

Code:
Using for Loop:



Code:

Using while Loop:



Analysis

Both loops produce the same output. The for loop is more concise and readable, while the while loop provides more control over iteration**.**

Task Description #3: Conditional Statements (Age Classification) Scenario

You are building a basic classification system based on age.

Task

• Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).

• Analyze the generated conditions and logic.

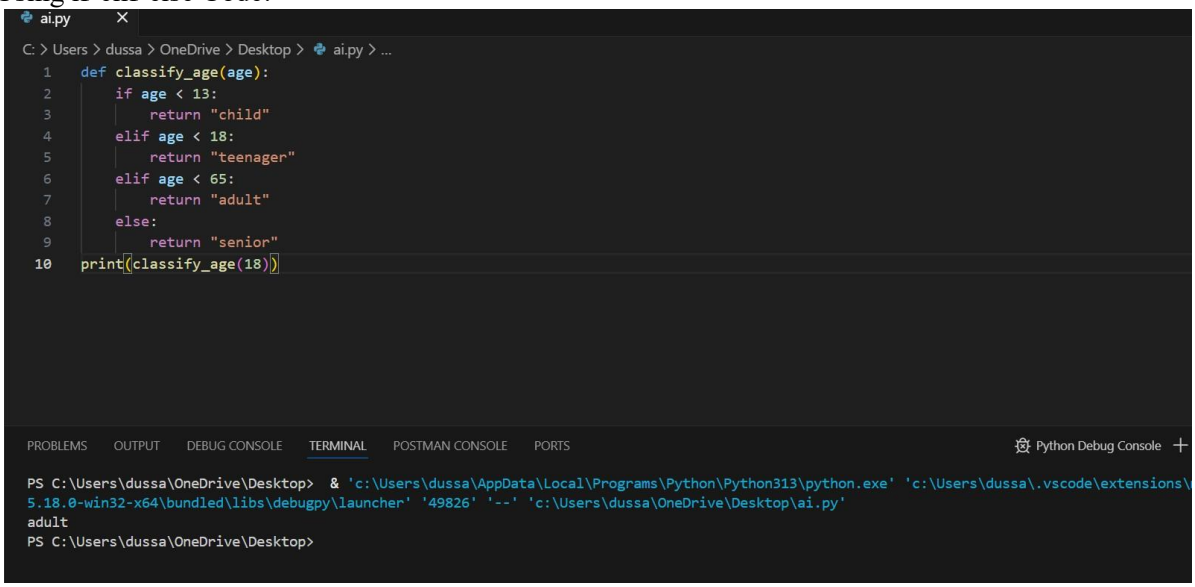
• Ask the AI to generate the same classification using alternative conditional structures (e.g.,
simplified conditions or dictionary-based logic).

Expected Output #3

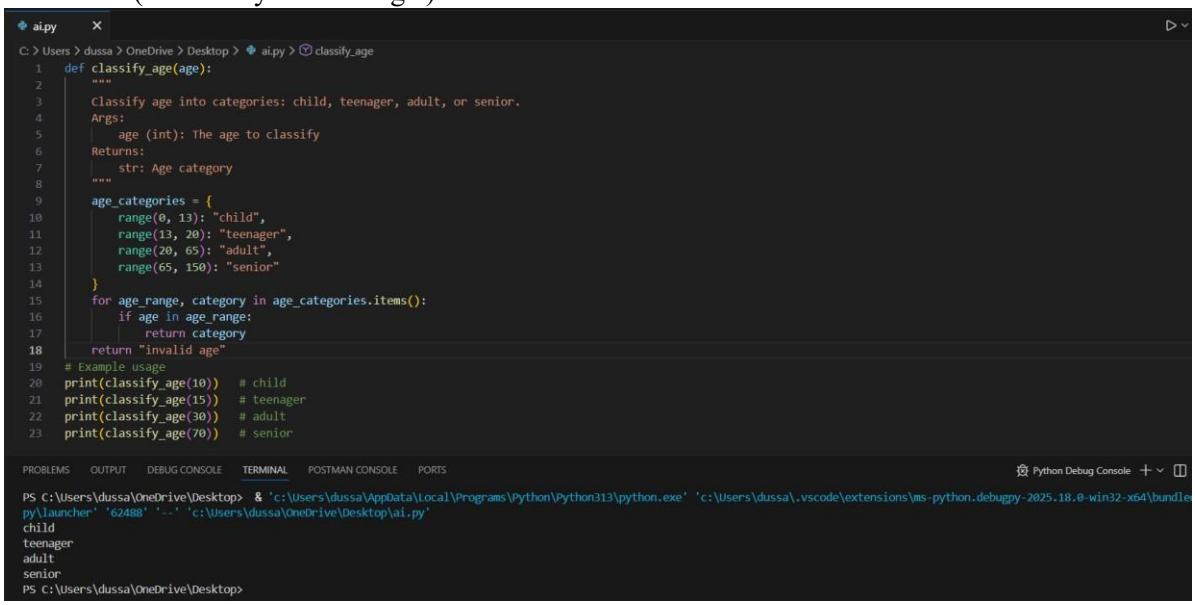
- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

Prompt:
Generate a Python function using if-elif-else to classify age as child, teenager, adult, or senior.

Using if-elif-else Code:



Alternative (Dictionary-Based Logic)



Analysis
The if-elif-else structure is simpler and more readable. The dictionary-based approach is creative but slightly less intuitive for beginners.

Task Description #4: For and While Loops (Sum of First n Numbers) Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a sum_to_n() function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

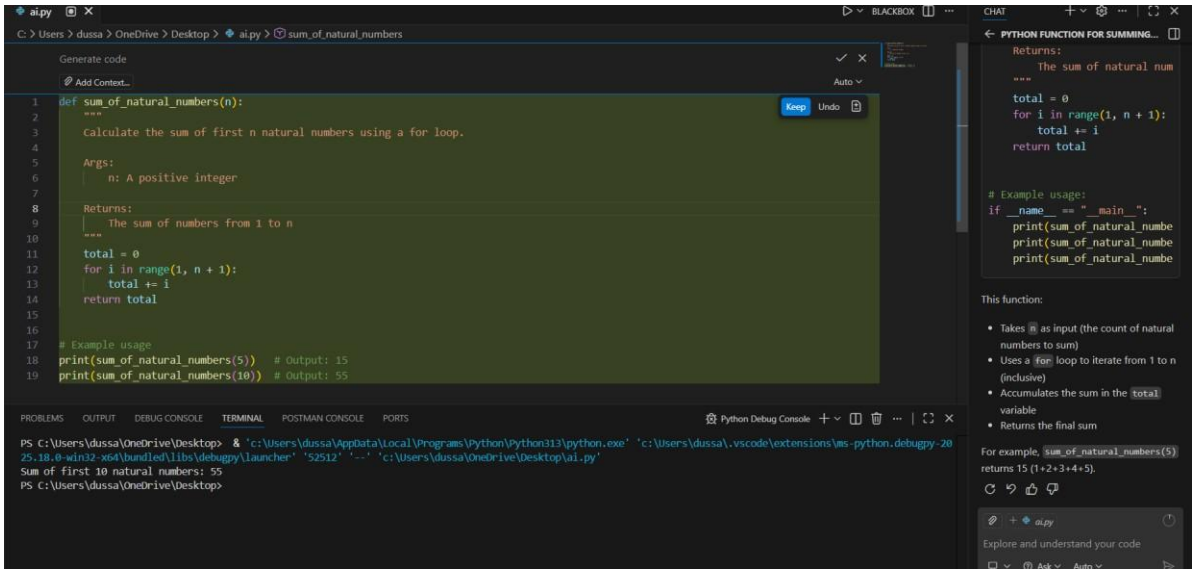
Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches.

Prompt:

Write a Python function to calculate the sum of first n natural numbers using a for loop.

Code:



Using Mathematical Formula

```
def sum_to_n_formula(n):
return n * (n + 1) // 2
```

Analysis

The formula-based approach is the most efficient with O(1) time complexity. Loop-based methods are easier to understand for beginners.

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

- Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

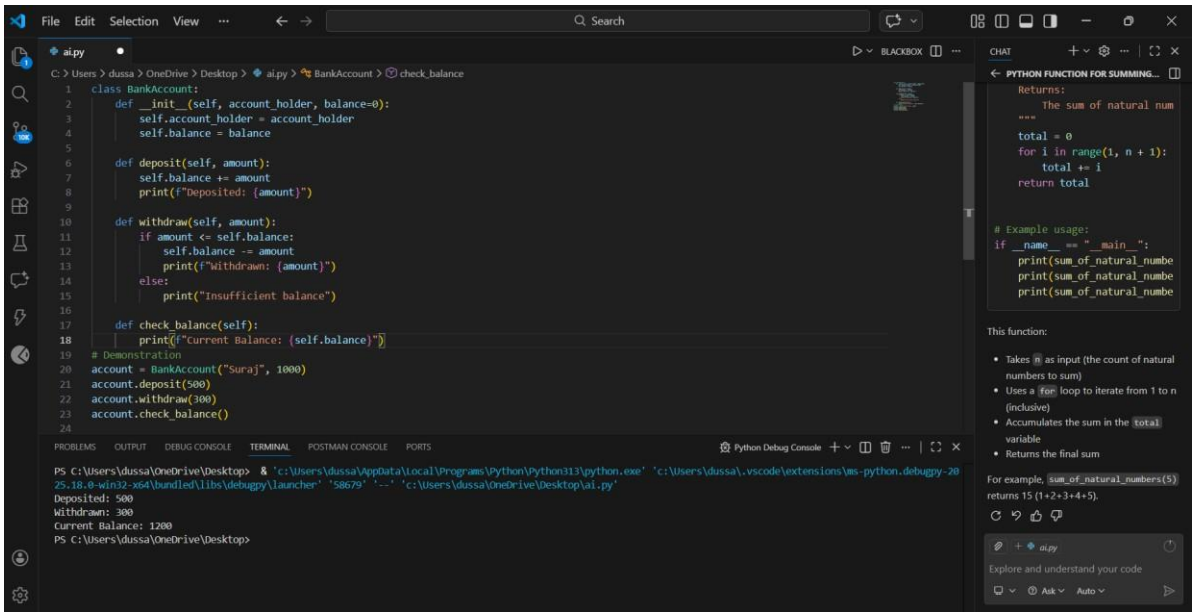
Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.

- Well-commented code with a clear explanation.

Prompt:
Generate a Python BankAccount class with deposit, withdraw, and check_balance methods.

Code:
AI-Generated Code with Comments



Explanation
The class correctly encapsulates account data and operations. Conditional checks prevent invalid withdrawals. The code is clean, readable, and suitable for a basic banking application.